Teaching Learning Evaluation Plan (TLEP)

Academic Year: 2026-2027 (ODD)

Name of the Department/ School	Computer Science & Engineering
Programme	Artificial Intelligence & Data Engineering
Year / Semester	II / Third
Course Code and Title	25PCCS201 - Operating System
Credit Value	3
Total Notional Hours Available	60 Hrs
LTPE Distribution	L: T: P: E:: 2:0:2:2
Assessment Model	70:30
Number of CA Components	7
Email ID	sujeet.sj@jainuniversity.ac.in
Faculty In-charge	Dr. Sujeet S. Jagtap

Faculty In-Charge **Course Coordinator** **Programme Head**

Program Outcomes

PO#	Statements
PO 1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop solutions of complex engineering problems that integrate socio-economic implications of technical applications.
PO 2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development (WK1 to WK4), applying epistemic skills that connect theoretical frameworks with real-world experiences.
PO 3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required (WK5).
PO 4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using critical inquiry methodologies and research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions (WK8), where stakeholders participate as co-creators in the knowledge development process.
PO 5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & AI based IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems (WK2 and WK6)
PO 6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment (WK1, WK5, and WK7), to drive social, economic, and political transformations.
PO 7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws (WK9), integrating equality, democratic behavior, tolerance, and associated living principles.
PO 8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams, applying conjoint communicated experience and democratic engagement.
PO 9	Communication: Communicate effectively and inclusively within the engineering community and society at large exhibiting emotional intelligence, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language and learning differences
PO 10	Project Management and Finance: Apply knowledge and understanding of engineering management principles, innovative problem-solving approaches and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary and entrepreneurial environments.
PO 11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change (WK8), contributing to human development. Knowledge and Attitude Profile (WK)

Program Specific Outcomes (PSO)

Program	CSE – AIML	
CSE – AIML	**PSO-1**	To design and develop innovative software and hardware solutions by applying principles of Computer Science and Engineering, addressing both industry and societal needs through cutting-edge technologies.
	PSO-2	To utilize modern tools and technologies in emerging domains to solve real-world problems, with a strong emphasis on ethical considerations and societal impact.
	PSO-3	To apply AI/ML engineering concepts of data processing, model development, and intelligent decision-making to develop safe, secure, and robust production-ready AI/ML solutions.

Knowledge and Attitude Profile

WK#	Statements
WK 1	A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.
WK 2	Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.
WK 3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.
WK 4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK 5	Knowledge, including efficient resource use, environmental impacts, whole-life cost, reuse of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.
WK 6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.
WK 7	Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development.
WK 8	Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.
WK 9	Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.

Year Wise Learning Outcomes: Second Year – Development Level

Dimension	LO #	Learning Outcome Statements
Critical Thinking	**LO1**	Challenge existing knowledge through structured analysis
	LO2	Apply systematic observation methods to engineering problems
	LO3	Evaluate technical information from multiple sources
	LO4	Develop reasoned arguments for engineering decisions and corroborate using AI
	LO5	Perceive objects and knowledge
	LO6	Raise doubts on status quo of objects and knowledge
Innovation and Professional Development	**LO1**	Execute project-based learning assignments independently
	LO2	Discuss and record observations of industrial visits
	LO3	Understand design thinking and its cultural and social implications
	LO4	Use Generative AI to visualize real-world problem
Research Experience	**LO1**	Conduct comprehensive literature reviews using AI-powered tools
	LO2	Define, identify, justify and articulate research problems using AI and develop basic research proposals
	LO3	Structure mini-projects using appropriate research methods
	LO4	Communicate effectively with industry experts and researchers related to relevant domain/ projects
Real World Exposure	**LO1**	Demonstrate interpersonal and social skills to constructively interact with society and industries
	LO2	Observe the real-world society, industry, market and insightfully infer
	LO3	Participate in group discussion and demonstrate social communication skills
	LO4	Depict the character required to practice togetherness and demonstrate team working skills
	LO5	Debate and depict tolerance on controversies and opposing ideas
	LO6	Engage meaningfully with industrial and societal partners
	LO7	Understand affective and cultural aspects of design thinking in engineering developments
	LO8	Demonstrate understanding of professional standards and professional ethics
	LO9	Show awareness of industrial practices and requirements
Youth Development	**LO1**	Practice social skills including displaying tendencies and attitudes towards cultural sensitivities, empathy and compassion by cultural activities, clubs and social events
	LO2	Interact effectively with general public, community leaders and NGOs
	LO3	Practice healthy food habits and healthy lifestyle habits leading to physical fitness
	LO4	Apply civic responsibilities in a given civic situation

Dimension	LO #	Learning Outcome Statements
	LO5	Practice civic and political conscience by abiding the constitution, laws and legislation, democratic behavior and ensuring social harmony
	LO6	Create awareness of social implications of engineering solutions using AI

Course Description

A. Prerequisite

Basic knowledge of Computer Organization, Programming Fundamentals, Data Structures, and Computer Networks is desirable for understanding operating system concepts, system-level programming, and resource management techniques.

B. Course Description

This course provides a comprehensive understanding of operating system principles, structures, services, and algorithms that manage computer system resources. It covers process management, CPU scheduling, threads, synchronization, deadlocks, memory management, virtual memory, file systems, disk scheduling, and I/O management. The course integrates theoretical concepts with hands-on Linux-based system programming, shell scripting, system calls, algorithm implementation, and system-level experimentation. Students also examine contemporary operating system architectures such as Linux, Android, and iOS.

Continuous assessment follows a seven-stage engineering progression — **Think → Measure → Engineer → Investigate → Observe → Build → Defend** — in which the first six assessments build on one another and CA-7 verifies individual understanding of the portfolio.

C. Impact / Application Areas

The knowledge and skills acquired through this course support the development and management of efficient computing systems in areas such as cloud computing, virtualization and containerization, embedded systems, mobile computing, cybersecurity, system software, and distributed computing. The course also provides a foundation for advanced study and professional practice in software systems and computing infrastructure.

Course Learning Objectives

The aim of this course is to:

1) Develop a systematic understanding of operating system structure, services, and process management, including CPU scheduling and thread execution.
2) Analyze process synchronization and deadlock scenarios to design correct and efficient concurrency control mechanisms.
3) Apply memory management, virtual memory, and file system techniques to evaluate and optimize system performance, reliability, and security.
4) Connect operating system concepts with real-world applications through hands-on programming assignments and mini-projects using Linux and open-source tools.

Course Outcomes

At the end of the course students will be able to:

CO #	Course Outcome statement	Blooms Level
CO 1	**Explain** operating system concepts, architecture, services, and resource-management functions in a computer system.	Understanding (L2)
CO 2	**Apply** process-management and CPU-scheduling techniques to determine process execution behaviour and scheduling performance.	Applying (L3)
CO 3	**Analyze** process synchronization, thread execution, and deadlock scenarios to identify concurrency and resource-allocation issues.	Analyzing (L4)
CO 4	**Analyze** memory-management, virtual-memory, and file-system techniques to determine their impact on system performance and resource utilization.	Analyzing (L4)
CO 5	**Evaluate** alternative scheduling, memory-management, synchronization, and file-system strategies based on performance, reliability, and security requirements.	Evaluating (L5)
CO 6	**Design and develop** operating-system-oriented solutions using Linux, shell scripting, and open-source tools for real-world computing scenarios.	Creating (L6)

CO-PO/PSO Mapping

COs\ POs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	2	1	-	-	-	-	-	-	-	-	-	1	2
CO2	3	3	-	2	2	-	-	-	-	-	-	1	3
CO3	3	3	1	3	2	-	-	-	-	-	-	1	3
CO4	3	3	-	3	2	-	-	-	-	-	-	1	3
CO5	3	3	2	2	2	-	-	-	-	-	1	1	3
CO6	3	2	3	2	3	-	-	2	2	2	2	2	3
Average	**3.00**	**2.50**	**2.00**	**2.40**	**2.20**	**-**	**-**	**2.00**	**2.00**	**2.00**	**1.50**	**1.17**	**2.8**

PO	Mapped COs and Strength	Rationale and Assessment Evidence
PO1	CO1-CO6	Operating-system architecture, scheduling, synchronization, memory, file systems, and Linux solution development directly apply computing and engineering knowledge.
PO2	CO1 (Low); CO2-CO5 (High); CO6 (Moderate)	Students identify system behaviour and analyze scheduling results, concurrency failures, memory/file-system performance, and solution requirements to reach substantiated conclusions.
PO3	CO3 (Low); CO5 (Moderate); CO6 (High)	Concurrency analysis informs solution formation; evaluation of alternatives supports design decisions; the Linux mini-project requires design and development of a working solution.
PO4	CO2 (Moderate); CO3-CO4 (High); CO5-CO6 (Moderate)	Scheduling simulations, execution traces, resource-allocation graphs, page-fault data, comparative evaluation, and project testing provide investigation and interpretation evidence.
PO5	CO2-CO5 (Moderate); CO6 (High)	Students use scheduling or memory simulators, Linux monitoring utilities, shell scripting, and open-source tools, while interpreting the limitations of selected techniques.

PO	Mapped COs and Strength	Rationale and Assessment Evidence
PO6 and PO7	Not directly mapped	The stated COs do not explicitly assess societal or environmental impact, sustainability, ethics, human values, or legal compliance; assigning a correlation would therefore be artificial.
PO8	CO6 (Moderate)	Pair programming, peer review, and mini-project execution provide assessable evidence of individual contribution and collaborative work.
PO9	CO6 (Moderate)	Project documentation, demonstration, and explanation of test evidence and limitations support technical written and oral communication.
PO10	CO6 (Moderate)	The mini-project requires planning tasks, managing time and resources, testing deliverables, and completing work within defined constraints.
PO11	CO5 (Low); CO6 (Moderate)	Evaluation of alternative strategies and independent use of Linux and open-source documentation develop adaptability and self-directed learning.

Pedagogical Approach and Learning Activities Aligned to the Continuous Assessments

CO #	Suggested Pedagogical Approach	Skill Acquired	Learning Activity (Brief Description)	Dimension Mapping	Learning Outcome Mapping
CO1	Interactive exposition with OS architecture diagrams and concept maps; progressive systems-thinking scenarios in which new evidence is revealed round by round.	Conceptual understanding; classification; architecture interpretation; structured analysis	CA-1 Systems Thinking Challenge: students identify the OS concept in a realistic scenario (e.g., overloaded computer lab, server with uneven core usage), model it, compare alternatives and defend a decision with AI corroboration.	**CT**	CT-LO1 – Challenge existing knowledge through structured analysis; CT-LO4 – Develop reasoned arguments and corroborate using AI.
CO2	Guided problem solving and simulation-based learning; hypothesis-driven performance experiments on CPU scheduling and process/thread behaviour; Linux process observation.	Algorithm application; systematic observation; measurement; performance interpretation	CA-2 Performance Investigation: students implement FCFS, SJF/SRTF, Priority and Round Robin, run controlled workloads (CPU-bound, short-burst, mixed), plot waiting/turnaround/response time and context switches, and recommend a policy.	**CT**	CT-LO2 – Apply systematic observation methods to engineering problems; CT-LO4 – Develop reasoned arguments and corroborate using AI.
CO3	Case-based inquiry, fault injection and trace analysis of race conditions, starvation and deadlocks; regression testing of fixes.	Concurrency analysis; root-cause diagnosis; solution design; validation	CA-3 Design & Debugging Challenge: students reproduce a defective scheduler/bank-account/two-lock-deadlock program, trace the failure, compare alternative fixes, implement one and prove it with boundary, stress and regression tests.	**CT, IP**	CT-LO1 – Challenge existing knowledge through structured analysis; IP-LO3 – Understand design thinking.
CO4	Problem-based, research-led investigation of memory management, page replacement, working set/thrashing and file/storage performance, supported by verified literature and reproducible experiments.	Literature evaluation; experimental design; data interpretation; research communication	CA-4 OS Research Mini-Project: students frame a measurable research question (e.g., page replacement under locality), review 3–5 sources, test a hypothesis and report evidence and threats to validity.	**CT**	CT-LO3 – Evaluate technical information from multiple sources; IP-LO4 – Use Generative AI to visualize real-world problems.

CO #	Suggested Pedagogical Approach	Skill Acquired	Learning Activity (Brief Description)	Dimension Mapping	Learning Outcome Mapping
CO5	Decision-oriented comparison of scheduling, synchronization, memory and file/disk strategies; trade-off analysis; adaptive viva (modify, break, diagnose, justify).	Multi-criteria evaluation; engineering judgement; technical argumentation	CA-1 decision stage, CA-2/CA-4 comparative evidence and CA-7 "Break My System" viva: students justify why an alternative was selected or rejected under stated constraints and workloads.	**CT**	CT-LO4 – Develop reasoned arguments and corroborate using AI; CT-LO6 – Raise doubts on status quo of objects and knowledge.
CO6	Project-based and experiential learning on a real Linux system and in a Rust Micro-OS (QEMU); milestone-based development, pair/team work, version control, testing and demonstration.	Linux and shell/Python automation; Rust systems programming; solution design; debugging; teamwork; documentation	CA-5 Linux Systems Engineering Challenge (baseline → intervention → validation using /proc, ps, top, vmstat, taskset, nice, strace) and CA-6 Rust Micro-OS Engineering Project (boot, tasks, Round Robin scheduler, timer, synchronization, allocator, specialization).	**IP, RWE**	IP-LO1 – Execute project-based assignments independently; IP-LO3 – Design thinking; RWE-LO4 – Teamwork; RWE-LO9 – Awareness of industrial practices.

Abbreviations: CT – Critical Thinking; IP – Innovation and Professional Development; RWE – Real World Exposure.

Session Wise Plan

Reference key: R1 – Silberschatz, Galvin and Gagne, Operating System Concepts; R2 – Tanenbaum and Bos, Modern Operating Systems; R3 – Linux manual pages and official command documentation. PD – Planned Date; AD – Actual Date. "CA Link" shows the continuous assessments that each session prepares students for.

Mod	Ses	PD	AD	Topic	Reading & References	Pedagogy / Activity Planned	Real World Exposure	CO	Mode of Delivery	CA Link
1	1	20/7/2026		Operating system purpose, functions, evolution, and types	R1 Ch. 1; R2 Ch. 1	Diagnostic questions, interactive exposition, and an OS function concept map.	Trace how a smartphone or ATM depends on OS resource management.	CO1	Classroom	CA-1
1	2			Computer-system organization: CPU, memory, I/O, interrupts, and storage hierarchy	R1 Ch. 1	Annotated architecture diagram and guided interrupt-sequence tracing.	Explain keyboard input and disk I/O as interrupt-driven events.	CO1	Classroom	CA-1
1	3			OS operations: user mode, kernel mode, timer, and protection	R1 Ch. 1-2	Think-Pair-Share using privilege and failure-isolation scenarios.	Relate dual-mode operation to isolating a crashed application.	CO1	Classroom	CA-1
1	4			Operating system services and user interfaces	R1 Ch. 2	Service-classification activity followed by a command-line versus GUI comparison.	Map file saving, printing, and network access to OS services.	CO1	Classroom	CA-1
1	5			System calls, APIs, parameters, and categories of system calls	R1 Ch. 2; R3 man pages	System-call tracing demonstration and guided interpretation of a short trace.	Observe open, read, write, fork, and exec calls with strace.	CO1	Demo/Lab	CA-1, CA-5
1	6			OS structures: monolithic, layered, modular, and microkernel designs	R1 Ch. 2; R2 Ch. 1	Jigsaw comparison; groups defend one architecture using stated criteria.	Compare Linux, Windows, and microkernel-based system structures.	CO1	Classroom	CA-1, CA-6
1	7			Virtual machines, hypervisors, and container fundamentals	R1 Ch. 18	Case-based comparison of virtual machines and containers.	Select an isolation approach for cloud deployment or a teaching lab.	CO1	Blended	CA-1, CA-4
1	8			Boot process, system programs, services, and daemons	R1 Ch. 2; R3 systemd docs	Guided Linux exploration of boot targets, services, and system utilities.	Inspect service status and startup behaviour using systemctl.	CO1	Lab	CA-5

Mod	Ses	PD	AD	Topic	Reading & References	Pedagogy / Activity Planned	Real World Exposure	CO	Mode of Delivery	CA Link
1	9			Module integration: OS architecture and service mapping	R1 Ch. 1-2	Retrieval quiz and OS Service and Architecture Map with peer feedback.	Diagnose which OS subsystem handles a realistic user request or fault.	CO1	Classroom	CA-1
2	10			Process concept, process states, PCB, and context switching	R1 Ch. 3	Process-state role play and state-transition diagram construction.	Inspect live process states and identifiers using ps and top.	CO2	Classroom/Lab	CA-1, CA-5
2	11			Process creation, execution, termination, and process trees	R1 Ch. 3; R3 fork/exec	Guided code walk-through of fork, exec, wait, and exit.	Create and inspect a parent-child process tree in Linux.	CO2	Demo/Lab	CA-3, CA-5
2	12			Interprocess communication: shared memory and message passing	R1 Ch. 3	Paired comparison and sequence-diagram activity for two IPC models.	Choose IPC for a producer service and a consumer application.	CO2	Classroom	CA-5
2	13			Threads, benefits, user/kernel threads, and multithreading models	R1 Ch. 4	Thread-execution tracing and model-comparison worksheet.	Relate multithreading to web servers and responsive applications.	CO2	Classroom	CA-2
2	14			CPU scheduling concepts, dispatcher, preemption, and evaluation criteria	R1 Ch. 5	Guided derivation of scheduling metrics using a common process set.	Connect response time, throughput, and fairness to interactive systems.	CO2	Classroom	CA-1, CA-2
2	15			FCFS, SJF, and SRTF scheduling algorithms	R1 Ch. 5	Worked examples followed by individual Gantt-chart problem solving.	Analyze convoy effect and short-job preference in batch workloads.	CO2	Classroom	CA-1, CA-2
2	16			Priority and Round Robin scheduling	R1 Ch. 5	Prediction-observation-comparison using different priorities and time quanta.	Tune a time quantum for an interactive multiuser workload.	CO2	Classroom	CA-2, CA-4
2	17			Multilevel queue and multilevel feedback queue scheduling	R1 Ch. 5	Scenario-based design of queues and feedback rules.	Model foreground, background, and system processes in separate queues.	CO2	Classroom	CA-2, CA-6
2	18			Comparative CPU-scheduling investigation and performance review	R1 Ch. 5	Simulation lab; calculate waiting, turnaround, and response times; interpret results.	Recommend a scheduling policy for batch, interactive, and real-time cases.	CO2	Lab	CA-2, CA-3
3	19			Concurrency, interleaving, race conditions, and data inconsistency	R1 Ch. 6	Trace-based inquiry using multiple execution interleavings.	Diagnose an incorrect shared counter in a multithreaded application.	CO3	Classroom	CA-3
3	20			Critical-section problem and correctness requirements	R1 Ch. 6	Claim-evidence-reasoning analysis of candidate critical-section solutions.	Assess mutual exclusion, progress, and bounded waiting in shared services.	CO3	Classroom	CA-3
3	21			Mutex locks, spinlocks, and hardware support for synchronization	R1 Ch. 6	Guided code tracing and lock-selection decision activity.	Compare busy waiting with blocking for short and long critical sections.	CO3	Classroom	CA-3, CA-6
3	22			Semaphores and semaphore-based synchronization	R1 Ch. 6	Incremental solution building with wait/signal trace tables.	Control access to a bounded pool of database connections.	CO3	Classroom	CA-3, CA-6
3	23			Producer-consumer and bounded-buffer problem	R1 Ch. 7	Collaborative design, dry run, and peer debugging of a semaphore solution.	Model data flow between a sensor producer and analytics consumer.	CO3	Classroom/Lab	CA-3, CA-5
3	24			Readers-writers and dining-philosophers problems	R1 Ch. 7	Small-group comparison of correctness, starvation, and fairness.	Relate readers-writers control to shared files and database records.	CO3	Classroom	CA-3
3	25			Monitors, condition variables, and language-level synchronization	R1 Ch. 7	Monitor pseudocode completion and peer explanation.	Connect condition variables to Java/POSIX thread coordination.	CO3	Classroom	CA-3
3	26			Deadlock characterization and resource-allocation graphs	R1 Ch. 8	Graph-based case analysis and deadlock identification.	Model competing processes requesting printers, files, and locks.	CO3	Classroom	CA-1, CA-3

Mod	Ses	PD	AD	Topic	Reading & References	Pedagogy / Activity Planned	Real World Exposure	CO	Mode of Delivery	CA Link
3	27			Deadlock prevention, avoidance, detection, and recovery	R1 Ch. 8	Banker's algorithm exercise and Concurrency and Deadlock Clinic.	Recommend a deadlock strategy for a database or transaction server.	CO3	Classroom/Lab	CA-1, CA-3
4	28			Memory-management background, address binding, and logical/physical addresses	R1 Ch. 9	Address-binding timeline and concept-check questions.	Relate compile, load, and execution time binding to application deployment.	CO4	Classroom	CA-6
4	29			Contiguous allocation, partitions, fragmentation, and compaction	R1 Ch. 9	Allocation simulation with first-fit, best-fit, and worst-fit.	Analyze fragmentation in a changing workload.	CO4	Classroom	CA-2, CA-6
4	30			Paging, frames, pages, address translation, and TLB	R1 Ch. 9	Guided numerical problems and address-translation diagramming.	Estimate effective access time with and without TLB hits.	CO4	Classroom	CA-6
4	31			Page-table structures and segmentation	R1 Ch. 9	Comparative worksheet and memory-layout interpretation.	Select a structure for sparse address spaces and modular programs.	CO4	Classroom	CA-4
4	32			Virtual memory and demand paging	R1 Ch. 10	Page-fault handling sequence and scenario-based questioning.	Explain how large applications execute with limited physical memory.	CO4	Classroom	CA-1, CA-4
4	33			FIFO, Optimal, and LRU page-replacement algorithms	R1 Ch. 10	Prediction and simulation using identical reference strings.	Compare page faults and observe Belady's anomaly.	CO4	Classroom/Lab	CA-2, CA-3
4	34			Frame allocation, working set, thrashing, and performance control	R1 Ch. 10	Performance-data interpretation and bottleneck diagnosis.	Use working-set evidence to respond to system thrashing.	CO4	Classroom	CA-1, CA-4
4	35			File concepts, access methods, directories, sharing, and protection	R1 Ch. 13	File-service scenario mapping and permission-analysis exercise.	Interpret ownership and permissions for shared Linux project files.	CO4	Classroom/Lab	CA-5
4	36			File-system implementation, allocation, and free-space management	R1 Ch. 14	Memory and File-System Performance Lab with an evidence-based improvement proposal.	Select file allocation and free-space methods for a stated workload.	CO4	Lab	CA-2, CA-5
5	37			Mass-storage structure and disk-scheduling algorithms	R1 Ch. 11	Head-movement simulation and comparative performance analysis.	Recommend FCFS, SSTF, SCAN, or C-SCAN for a storage workload.	CO5	Classroom	CA-2, CA-4
5	38			I/O system, device drivers, buffering, caching, and spooling	R1 Ch. 12	Device-I/O flow mapping and bottleneck case discussion.	Analyze print spooling and buffered network or disk I/O.	CO5	Classroom	CA-1, CA-5
5	39			Protection, access control, authentication, and OS security principles	R1 Ch. 16-17	Threat-scenario analysis and least-privilege decision activity.	Review permissions and privilege boundaries on a multiuser server.	CO5	Classroom	CA-5, CA-6
5	40			Linux process, memory, storage, and performance-monitoring tools	R3 Linux man pages	Guided hands-on investigation using ps, top, free, vmstat, df, du, and iostat.	Collect and interpret live resource-utilization evidence.	CO6	Lab	CA-5
5	41			Shell scripting foundations: variables, conditions, loops, and commands	R3 Bash manual	Live coding, predict-run-explain cycles, and short scripting exercises.	Automate process checks, file handling, and log inspection.	CO6	Lab	CA-5
5	42			Shell-based system administration and resource automation	R3 Bash and cron docs	Paired scripting challenge with test cases and peer code review.	Build a backup, disk-usage alert, or service-health utility.	CO6	Lab	CA-5
5	43			Evaluation of OS strategies for a specified system	R1 selected chapters	OS Strategy Review Board using criteria, decision matrix, and oral defence.	Recommend a configuration for a server, embedded, or safety-critical case.	CO5	Seminar	CA-1, CA-7

Mod	Ses	PD	AD	Topic	Reading & References	Pedagogy / Activity Planned	Real World Exposure	CO	Mode of Delivery	CA Link
5	44			Operating-system mini-project development and iterative testing	R3 relevant docs	Project-based learning, pair programming, debugging, and instructor review.	Develop and test a Linux tool using realistic inputs and constraints.	CO6	Project/Lab	CA-6
5	45			Mini-project demonstration, reflection, and course integration	Project documentation	Product demonstration, rubric-based peer feedback, and reflective discussion.	Communicate solution value, limitations, test evidence, and improvements.	CO6	Project/Demo	CA-6, CA-7

Assessment Strategy Design

Overall Assessment Distribution

Assessment Type	Components	Marks	Purpose
Formative – Reasoning & Evidence	CA-1 Systems Thinking Challenge (10) + CA-2 Performance Investigation (10) + CA-3 Design & Debugging Challenge (10)	30	Builds scenario reasoning, experimental measurement and debugging skill; each CA is evidence-based and rubric-assessed (5 criteria).
Formative + Portfolio – Research	CA-4 OS Research Mini-Project	10	Develops literature evaluation, research questioning, reproducible experimentation and research-style reporting.
Formative + Real-World – Systems Engineering	CA-5 Linux Systems Engineering Challenge (10) + CA-6 Rust Micro-OS Engineering Project (15)	25	Moves from observing a real OS to implementing selected mechanisms; evidences tool use, teamwork and professional practice.
Formative – Defence	CA-7 OS Engineering Viva & Demonstration	5	Verifies individual understanding, contribution and responsible AI use across the entire portfolio.
Summative (SEE)	Semester End Examination (written, all units)	30	Final comprehensive evaluation of OS concepts and analytical problem solving.
Total	**CIE 70 + SEE 30**	**100**	**Assessment model 70:30**

CA Component Summary

CA	Assessment	Marks	Core Capability	Mode / Format	Language & Tools	Principal Evidence	Window
CA-1	Systems Thinking Challenge	10	Think	Scenario-based progressive challenge; individual or team	Reasoning-led; Python/Linux evidence optional	Engineering decision document: model, comparison, justification, AI corroboration	Weeks 4–6
CA-2	OS Performance Investigation	10	Measure	Controlled experiment on a common framework with different investigation questions; individual or team	Python (default); Bash; Rust optional	Technical report, source code, raw measurements, plots	Weeks 6–8
CA-3	OS Design & Debugging Challenge	10	Engineer	Defective-implementation, fault-injection or requirement-only challenges; individual diagnosis, team design	Python (default); Bash/Linux; Rust for advanced challenges	Failure → fix → retest evidence, test cases, regression results	Weeks 8–10
CA-4	OS Research Mini-Project	10	Investigate	Research proposal → faculty approval → literature matrix → experiment → report; individual, pair or team of 3	Python (default); Linux/Bash; Rust for systems-level questions	Research proposal, literature matrix, source code, raw data, report, short presentation	Weeks 8–11
CA-5	Linux Systems Engineering Challenge	10	Observe	Real-Linux instrumentation and intervention challenge; baseline → intervention → validation; individual or team	Bash, Python, /proc, standard Linux tools; Rust optional	Command outputs, /proc observations, before/after measurements, source code, environment record	Weeks 10–13

CA	Assessment	Marks	Core Capability	Mode / Format	Language & Tools	Principal Evidence	Window
CA-6	Rust Micro-OS Engineering Project	15	Build	Core + specialization model; team project with individual contribution statements and viva; QEMU-based	Rust (kernel); Bash/Cargo (build); Python (trace analysis)	Repository, architecture and design documents, traces, test results, performance data, report, demonstration	Weeks 9–14
CA-7	OS Engineering Viva & Demonstration	5	Defend	Adaptive, evidence-based individual viva (8–12 min per student): Demonstrate → Explain → Modify → Break → Diagnose → Justify → Extend	Student portfolio artefacts from CA-1 to CA-6	Live demonstration, oral defence, contribution log, viva record sheet	Week 15

Formative Assessment (Continuous)

Assessment Tool	Weightage	Frequency	COs Assessed	Individualization
CA-1: Systems Thinking Challenge	10	1 (progressive, multi-round)	CO1–CO5	Individual with unique scenario/parameters, or team of 3–4
CA-2: OS Performance Investigation	10	1 experiment	CO2–CO6	Common framework, different investigation question per student/team
CA-3: OS Design & Debugging Challenge	10	1 (rotating subsystem)	CO2–CO6	Individual diagnosis; team design challenges
CA-4: OS Research Mini-Project	10	1 (4-week plan)	CO1–CO6	Individual, pair or team of 3; identifiable contribution per member
CA-5: Linux Systems Engineering Challenge	10	1 (4-week plan)	CO2–CO6	Individual or team; different challenge per student/team
CA-6: Rust Micro-OS Engineering Project	15	1 (6-week plan)	CO1–CO6	Team; individual contribution statement and viva
CA-7: OS Engineering Viva & Demonstration	5	1 (8–12 min each)	CO1–CO6	Individual; adaptive questions from CO-mapped pool
Total CIE	**70**			

A. CA-1: Systems Thinking Challenge (10 marks)

Mode: Scenario-based progressive challenge; individual or team. **Timeline:** Weeks 4–6; due Week 6 (24–29 Aug 2026).

Students analyse a realistic computing scenario (e.g., overloaded computer lab, starvation in a priority scheduler, high page-fault rate), moving through Identify → Model → Analyse alternatives → Select → Justify. Scenarios are released in rounds so that new evidence must be absorbed and the decision revised.

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
Problem identification & OS concept recognition: names the OS subsystem, explains why the scenario represents that problem and separates relevant concepts from distractions	Stage 1 – Identify the OS concept involved	CT-LO1 (primary); RWE-LO2 (supports)	CO1 (primary); CO2–CO5 by scenario	2
Problem modelling: process table/Gantt chart, resource-allocation graph, page-reference string, disk queue or equivalent, with a brief explanation	Stage 2 – Model the problem	CT-LO2 (primary)	CO2–CO4 by scenario	2

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
Analysis of alternatives: at least two plausible mechanisms compared on measurable criteria (waiting, turnaround, response time, page faults, head movement, fairness)	Stage 3 – Analyse alternatives	CT-LO1, CT-LO3 (primary); IP-LO3 (supports)	CO2–CO5	2
Selection of appropriate mechanism: explicit decision with selection criteria, constraints and expected outcome	Stage 4 – Select a mechanism	CT-LO4 (primary)	CO5	2
Engineering justification & communication: claim → evidence → interpretation → decision; trade-offs, assumptions, limitations, conditions for reconsideration; AI use disclosed and verified	Stage 5 – Justify the decision; AI corroboration	CT-LO4 (primary); CT-LO3 (supports)	CO1, CO5	2
Subtotal				**10**

B. CA-2: OS Performance Investigation (10 marks)

Mode: Controlled experiment on a common framework with different investigation questions; individual or team. **Timeline:** Weeks 6–8; due Week 8 (7–12 Sep 2026).

Students implement OS mechanisms (CPU scheduling, Round Robin quantum, process vs thread cost, synchronization contention, page replacement, memory allocation, disk scheduling, Linux process monitoring), run controlled workloads and draw evidence-based engineering conclusions: Question → Hypothesis → Experiment → Measurement → Interpretation → Decision.

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
Experimental design & methodology: testable performance question, hypothesis, independent/dependent/controlled variables, workloads, repetitions	Steps 1–3 – Question, hypothesis, variables	CT-LO2 (primary); IP-LO1 (supports)	CO2–CO5 by experiment	2
Implementation & correctness: correct, tested implementation of the OS mechanisms in the stated language	Step 4 – Implement the experiment	IP-LO1 (primary)	CO2–CO6 by experiment	2
Data collection & measurement: systematic, reproducible measurements; ≥3 repetitions where timing is involved; environment documented; noise explained	Steps 5–6 – Execute and collect data	CT-LO2 (primary); RWE-LO9 (supports)	CO2–CO5	2
Analysis & visualization: tables and plots with interpretation; workload-dependence of results	Step 7 – Analyse and visualize	CT-LO3 (primary); IP-LO4 (supports)	CO4, CO5	2
Engineering interpretation & conclusion: hypothesis tested, trade-offs, limitations, context-specific recommendation (not metric-chasing)	Steps 8–9 – Interpret; recommend	CT-LO4 (primary); CT-LO1 (supports)	CO5	2
Subtotal				**10**

C. CA-3: OS Design & Debugging Challenge (10 marks)

Mode: Defective-implementation, fault-injection or requirement-only challenges; individual diagnosis, team design. **Timeline:** Weeks 8–10; due Week 10 (21–26 Sep 2026).

Students receive a defective or requirement-only OS-related implementation (e.g., incorrect Round Robin, starvation, bank-account race, two-lock deadlock, Banker's defect, first-fit error, LRU error, zombie processes) and follow Observe → Identify concept → Model → Root cause → Alternatives → Implement → Prove the fix.

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
Problem diagnosis & root cause: separates symptom, immediate cause and violated OS principle; failure is reproduced	Stages 1, 2, 4 – Observe, identify, root cause	CT-LO1 (primary); CT-LO2 (supports)	CO2–CO4 by challenge	2

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
System/algorithm modelling & reasoning: trace, state diagram, resource graph, Gantt chart or memory map of the failing execution	Stage 3 – Model the failure	CT-LO2 (primary)	CO2–CO4 by challenge	2
Solution design & alternative analysis: at least two corrective approaches compared on correctness, performance, complexity and fairness	Stage 5 – Analyse alternative fixes	IP-LO3 (primary); CT-LO1 (supports)	CO5	2
Implementation & validation: corrected implementation; normal, boundary, stress, adversarial and recovery tests; before/after regression comparison	Stages 6–7 – Implement and prove	IP-LO1 (primary)	CO3, CO6	2
Technical defence, trade-offs & communication: overhead, reduced concurrency, complexity; AI record with verification	Required response format §10–12	CT-LO4 (primary); RWE-LO9 (supports)	CO5	2
Subtotal				**10**

D. CA-4: OS Research Mini-Project (10 marks)

Mode: Research proposal → faculty approval → literature matrix → experiment → report; individual, pair or team of 3. **Timeline:** Weeks 8–11; due Week 11 (28 Sep–3 Oct 2026).

Students formulate a focused, measurable OS research question (from the topic bank, e.g., Round Robin quantum, fairness vs throughput, lock contention, page replacement under locality, CPU affinity), review 3–5 verified sources, identify a gap, test a hypothesis experimentally and report limitations. Research Experience outcomes are developed here and credited under CT-LO3 for this course.

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
Research problem, question & gap: specific, measurable question narrower than the topic; plausible research gap	Stages 1, 3, 4 – Problem, gap, question	CT-LO1, CT-LO6 (primary)	CO1 (primary); CO2–CO5	2
Literature review & research justification: 3–5 credible references (≥2 papers where available) verified against originals; literature matrix	Stage 2 – Literature discovery	CT-LO3 (primary)	CO1	1
Experimental methodology & design: hypothesis, variables, baseline, repetitions, reproducibility (OS, versions, seed)	Stages 5–6 – Hypothesis, methodology	CT-LO2 (primary); IP-LO1 (supports)	CO2–CO5	2
Prototype/implementation & correctness: simulator, benchmark or Linux experiment in the stated language	Stage 7 – Prototype/experiment	IP-LO1 (primary)	CO6	1
Data collection, analysis & visualization: numerical evidence (not screenshots), plots, statistical summaries with explanation	Stage 8 – Results	CT-LO3 (primary); IP-LO4 (supports)	CO4, CO5	2
Research interpretation, limitations & conclusion: answers the question using OS concepts; threats to validity; future work	Stages 9–10 – Interpretation, conclusion	CT-LO4, CT-LO6 (primary)	CO5	1
Technical communication & AI-use disclosure: research-style report; mandatory disclosure statement; integrity and citations	Report #15 and #12 AI protocol	RWE-LO8 (primary); CT-LO4 (supports)	CO1	1
Subtotal				**10**

E. CA-5: Linux Systems Engineering Challenge (10 marks)

Mode: Real-Linux instrumentation and intervention challenge; baseline → intervention → validation; individual or team. **Timeline:** Weeks 10–13; due Week 13 (12–17 Oct 2026).

Students observe, control and instrument real OS behaviour on Linux (process-state investigator, priority and CPU-affinity experiments, IPC, memory growth, file/I-O patterns, resource limits, integrated incident investigation) using ps, top, /proc, vmstat, iostat, taskset, nice and strace, automated with Python/Bash, under strict laboratory safety rules.

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
Problem understanding & OS mechanism identification: focused Linux/OS question, correct mechanism, hypothesis	Framework steps 1–2 – Observe, formulate	CT-LO5 (primary); CT-LO1 (supports)	CO1, CO2	2
Linux instrumentation & tool usage: appropriate tools and meaningful measurements; environment (distribution, kernel, CPU, RAM, versions) recorded	Step 3 – Instrument	CT-LO2 (primary); IP-LO1 (supports)	CO6	2
Implementation / engineering intervention: justified change to priority, affinity, workload, program or configuration	Steps 4, 5, 7 – Experiment, control, engineer	IP-LO3 (primary); IP-LO1 (supports)	CO6; CO2–CO4 by task	2
Experimental evidence & analysis: baseline vs intervention, quantitative comparison, repeated runs, unexpected behaviour explained	Steps 6 – Analyse; evidence rule	CT-LO2, CT-LO3 (primary)	CO2–CO4 by task	2
Validation, technical explanation & defence: reproducible result; safety rules followed; link to theory and to a future kernel-level implementation	Steps 8–9 – Validate, defend	CT-LO4 (primary); RWE-LO8, RWE-LO9 (supports)	CO6	2
Subtotal				**10**

F. CA-6: Rust Micro-OS Engineering Project (15 marks)

Mode: Core + specialization model; team project with individual contribution statements and viva; QEMU-based. **Timeline:** Weeks 9–14; due Week 14 (19–24 Oct 2026).

Teams build a small educational Micro-OS in Rust in incremental layers — boot, console/diagnostics, tasks, ready queue, Round Robin scheduler, timer-driven scheduling, synchronization, memory allocator, integration — plus one specialization track (scheduler, memory, concurrency, system interface, storage, security or research-to-kernel). Evidence must show what each mechanism is doing.

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
Architecture & OS mechanism design: modular design with data/control-flow diagram and justified decisions	Layers 1–10; architecture diagram requirement	IP-LO3 (primary); CT-LO4 (supports)	CO1, CO6	2
Rust implementation quality: correct, idiomatic, controlled low-level code; justified and isolated `unsafe` with stated invariants	Rust and `unsafe` policy	IP-LO1 (primary)	CO6	4
Core OS functionality: boot, tasks, Round Robin scheduler, timer, synchronization and allocator working and integrated	Core requirements table	IP-LO1 (primary)	CO2, CO3, CO4	3
Testing, debugging & validation: TC1–TC6, unit/mechanism/integration tests, debugging log, repeatability	Testing strategy; debugging strategy	CT-LO2 (primary); RWE-LO9 (supports)	CO6	2
Performance / evidence-based evaluation: scheduler traces, switch counts, latency, memory statistics; Python trace analysis; Linux/theory comparison	Performance evaluation; Python analysis layer	CT-LO3, CT-LO4 (primary)	CO2–CO5	2
Innovation / specialization: advanced component in the chosen track	Specialization tracks A–G	IP-LO3, IP-LO4 (primary)	CO6; track-specific CO	1

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
Documentation & technical demonstration: report, version-controlled repository, individual contribution statement, demo	Final report; submission package	RWE-LO3, RWE-LO4 (supports); RWE-LO9 (supports)	CO6	1
Subtotal				**15**

G. CA-7: OS Engineering Viva & Demonstration (5 marks)

Mode: Adaptive, evidence-based individual viva (8–12 min per student): Demonstrate → Explain → Modify → Break → Diagnose → Justify → Extend. **Timeline:** Week 15; due Week 15 (26–31 Oct 2026).

Each student demonstrates at least one artefact from the CA portfolio and defends it through scenario-based questioning, a controlled "break my system" sequence, AI-use questions and verification of individual contribution. Questions are drawn from a common CO-mapped question pool.

Item (rubric criterion and evidence assessed)	Framework basis (guide)	LO mapping	CO	Marks
Technical demonstration: artefact runs and the student explains what is being observed	Demonstration (2 min)	CT-LO2 (primary)	CO6 (artefact-specific COs)	1
Conceptual understanding: links implementation to OS theory (process → PCB/state, scheduling → ready queue, synchronization → critical section, memory → virtual/physical abstraction)	Explanation and technical questioning	CT-LO5 (primary)	CO1–CO6 by question	1
Engineering reasoning & trade-offs: justifies approach, quantum, synchronization, allocator, language and tool choices	Technical questioning; justify/extend levels	CT-LO4 (primary)	CO5	1
Debugging / scenario-based questioning: reasons systematically about a controlled hypothetical or modification	Challenge/diagnosis (2 min)	CT-LO1, CT-LO6 (primary)	CO3, CO6 and artefact-specific COs	1
Communication, individual contribution & defence: clarity, independence, acknowledgement of limitations, responsible-AI answers	Defence / future extension	RWE-LO3 (primary); RWE-LO4, IP-LO1 (supports)	CO1–CO6	1
Subtotal				**5**

Rubric performance levels (Basic → Exceptional, or Excellent/Good/Developing/Needs Improvement) are applied as defined in each CA framework document; each criterion is scored on the marks shown above.

Summative Assessment (Major Evaluations)

Component	Weightage	Duration	Question Pattern	COs Assessed
End Semester Assessment	30	2hrs	Decided by the PAC of the department	CO1-CO6

Section	Format	Cognitive level	Marks
Part A	6 short-answer questions × 2 marks (concepts, definitions, quick numerical)	Remember / Understand / Apply (about 40%)	12
Part B	3 questions × 6 marks, on scheduling and synchronization, memory and file systems, and modern OS case scenarios	Analyse / Evaluate / Create (about 60%)	18
Total	**Covers all units (CO1–CO6)**		**30**

Overall Assessment

Assessment type	What it covers	Marks
Formative (CA)	CA-1 to CA-3 (reasoning, performance, debugging) and CA-7 (viva)	35
Portfolio / Research	CA-4 research proposal, literature matrix, raw data, report; AI-use records across CA-1 to CA-6	10
Real-World Application	CA-5 Linux engineering evidence and CA-6 Rust Micro-OS project, demonstration and contribution records	25
Summative (SEE)	Written examination	30
Total		**100**

Assessment Summary

BoS-Approved Dimensions for this Course

- **Primary Dimension:** Critical Thinking (CT) – **60%**
- **Secondary Dimension:** Innovation & Professional Development (IN) – **30%**
- **Tertiary Dimension:** Real World Exposure (RWE) – **10%**

Research Experience outcomes (literature review, research question, methodology) are developed mainly in CA-4 and are credited under CT-LO3 and CT-LO4, because Research Experience is not a BoS-approved dimension for this course.

Dimension Distribution Across Components

Component	Marks	Primary % (CT)	Primary Marks	Secondary % (IN)	Secondary Marks	Supporting % (RWE)	Supporting Marks
CA-1: Systems Thinking Challenge	10	75%	7.5	15%	1.5	10%	1
CA-2: OS Performance Investigation	10	65%	6.5	30%	3	5%	0.5
CA-3: OS Design & Debugging Challenge	10	80%	8	15%	1.5	5%	0.5
CA-4: OS Research Mini-Project	10	70%	7	20%	2	10%	1
CA-5: Linux Systems Engineering Challenge	10	30%	3	50%	5	20%	2
CA-6: Rust Micro-OS Engineering Project	15	40%	6	50%	7.5	10%	1.5
CA-7: OS Engineering Viva & Demonstration	5	80%	4	10%	0.5	10%	0.5
Total CIE	**70**	**60%**	**42**	**30%**	**21**	**10%**	**7**
Semester End Examination (SEE)	30	60%	18	30%	9	10%	3
TOTAL	**100**	**60%**	**60**	**30%**	**30**	**10%**	**10**

The per-CA allocation apportions each criterion's marks across dimensions in line with the primary/secondary emphasis stated in the CA framework documents, calibrated so that the course total equals the BoS-approved 60:30:10 ratio.

Component Overview Table

Component	Type	Format	Marks	Weeks	Dimension Focus	Hours*
CA-1: Systems Thinking Challenge	Assignment / Case analysis	Individual or team	10	Weeks 4–6	CT 75% + IN 15% + RWE 10%	4
CA-2: OS Performance Investigation	Practical / Report	Individual or team (2–3)	10	Weeks 6–8	CT 65% + IN 30% + RWE 5%	6
CA-3: OS Design & Debugging Challenge	Practical / Design project	Individual and team	10	Weeks 8–10	CT 80% + IN 15% + RWE 5%	6
CA-4: OS Research Mini-Project	Project (Research)	Individual / pair / team of 3	10	Weeks 8–11	CT 70% + IN 20% + RWE 10%	8
CA-5: Linux Systems Engineering Challenge	Practical (Linux)	Individual or team	10	Weeks 10–13	CT 30% + IN 50% + RWE 20%	8
CA-6: Rust Micro-OS Engineering Project	Project (Applied / Group)	Team with individual contribution	15	Weeks 9–14	CT 40% + IN 50% + RWE 10%	20
CA-7: OS Engineering Viva & Demonstration	Presentation (demonstration) + Viva	Individual	5	Week 15	CT 80% + IN 10% + RWE 10%	2
ESA: Semester End Examination (SEE)	Written	Individual	30	Week 16	CT 60% + IN 30% + RWE 10%	2
TOTAL	**–**	**–**	**100**	**–**	**CT 60%, IN 30%, RWE 10%**	**–**

**Indicative evaluation/assessment hours; to be confirmed by the faculty in-charge. Format options: Individual (CA-7, SEE); Individual or team (CA-1, CA-2, CA-3, CA-4, CA-5); Team with individual contribution (CA-6). Delivery modes: continuous, face-to-face, lab and online submission.*

Assessment Calendar

Sl. No.	Assessment Week / Date (Tentative)	Assessment Instrument	Marks	Description	COs Covered
1	Weeks 4–6; due Week 6 (24–29 Aug 2026)	CA-1: Systems Thinking Challenge	10	Progressive scenario challenge in rounds (identify → model → compare → select → justify) with AI-corroboration record.	CO1–CO5
2	Weeks 6–8; due Week 8 (7–12 Sep 2026)	CA-2: OS Performance Investigation	10	Hypothesis-driven experiment on scheduling/threads/synchronization/memory; report with raw data and plots.	CO2–CO6
3	Weeks 8–10; due Week 10 (21–26 Sep 2026)	CA-3: OS Design & Debugging Challenge	10	Diagnose, redesign and validate a defective OS mechanism; failure → fix → retest evidence and regression tests.	CO2–CO6
4	Weeks 8–11; proposal and approval Week 8; due Week 11 (28 Sep–3 Oct 2026)	CA-4: OS Research Mini-Project	10	Research proposal and faculty checkpoint (Wk 8), methodology and prototype (Wk 9), experiments (Wk 10), analysis, report and presentation (Wk 11).	CO1–CO6
5	Weeks 10–13; due Week 13 (12–17 Oct 2026)	CA-5: Linux Systems Engineering Challenge	10	Toolkit and problem statement (Wk 10), baseline (Wk 11), intervention and validation (Wk 12), analysis, report and demo (Wk 13).	CO2–CO6
6	Weeks 9–14; milestone reviews Wks 10, 12; demo Week 14 (19–24 Oct 2026)	CA-6: Rust Micro-OS Engineering Project	15	Boot and diagnostics → tasks and scheduler → timer, synchronization, memory → specialization and integration → testing, report and demonstration.	CO1–CO6
7	Week 15 (26–31 Oct 2026)	CA-7: OS Engineering Viva & Demonstration	5	Individual adaptive viva with demonstration of one portfolio artefact (8–12 min).	CO1–CO6
		Total (Formative Assessment)	**70**		**CO1–CO6**
8	Week 16 (2–7 Nov 2026)	Semester End Examination (SEE)	30	Written examination covering all units.	CO1–CO6
		Grand Total	**100**		

Course Outcomes (CO) to Components Mapping

CO	CA-1	CA-2	CA-3	CA-4	CA-5	CA-6	CA-7	SEE	Primary Component	Evidence
CO1	✓✓			✓	✓	✓	✓✓	✓✓	CA-1, CA-7, SEE	Decision documents; viva; written examination
CO2	✓	✓✓	✓✓	✓✓	✓✓	✓✓	✓	✓✓	CA-2, CA-3, CA-5, CA-6	Scheduling experiments; process investigation; scheduler trace
CO3	✓	✓✓	✓✓	✓✓	✓✓	✓✓	✓✓	✓✓	CA-3, CA-5, CA-6, CA-7	Race/deadlock debugging; contention experiments; kernel lock
CO4	✓	✓✓	✓	✓✓	✓✓	✓✓	✓	✓✓	CA-2, CA-4, CA-5, CA-6	Page-replacement/allocation data; memory investigation; allocator
CO5	✓✓	✓✓	✓	✓✓	✓	✓	✓✓	✓✓	CA-1, CA-2, CA-4, CA-7	Alternatives analysis; comparative experiments; trade-off defence
CO6	✓	✓	✓	✓	✓✓	✓✓	✓✓	✓	CA-5, CA-6, CA-7	Linux engineering evidence; Rust Micro-OS; demonstration

Legend: ✓✓ = Primary/direct assessment | ✓ = Supporting/reinforcing

Year-Level Learning Outcomes Coverage

Primary dimension: Critical Thinking (CT); secondary: Innovation & Professional Development (IP); tertiary: Real World Exposure (RWE). Only learning outcomes with direct evidence in the CA frameworks are marked.

Learning Outcome	CA-1	CA-2	CA-3	CA-4	CA-5	CA-6	CA-7	ESA
CT-LO1 – Challenge existing knowledge through structured analysis	✓		✓				✓	✓
CT-LO2 – Apply systematic observation methods to engineering problems	✓	✓	✓	✓	✓	✓	✓	
CT-LO3 – Evaluate technical information from multiple sources	✓	✓		✓	✓	✓		
CT-LO4 – Develop reasoned arguments and corroborate using AI	✓	✓	✓	✓	✓	✓	✓	✓
CT-LO5 – Perceive objects and knowledge					✓		✓	
CT-LO6 – Raise doubts on status quo of objects and knowledge				✓			✓	
IP-LO1 – Execute project-based learning assignments independently		✓	✓	✓	✓	✓	✓	
IP-LO2 – Discuss and record observations of industrial practice					✓	✓		
IP-LO3 – Understand design thinking and its implications	✓		✓		✓	✓		
IP-LO4 – Use Generative AI to visualize real-world problems		✓		✓	✓	✓		
RWE-LO2 – Observe real-world society/industry and infer	✓							
RWE-LO3 – Group discussion and social communication skills						✓	✓	
RWE-LO4 – Teamwork and togetherness				✓	✓	✓	✓	
RWE-LO8 – Professional standards and ethics				✓	✓	✓	✓	
RWE-LO9 – Awareness of industrial practices and requirements	✓	✓	✓		✓	✓		

AI Integration Strategy

AI Policy Levels

Level 1 – PROHIBITED: No AI tools permitted; requires authentic personal work (reflective writing, experiential participation, skill demonstrations).

Level 2 – PERMITTED (with disclosure): AI tools allowed for specific purposes; must be disclosed in the submission (research, framework analysis, preliminary drafting, data exploration).

Level 3 – ENCOURAGED (with disclosure): AI actively promoted as a learning tool; use must be documented (data analysis, visualization, exploring perspectives, creative ideation).

Component	Policy Level	Rationale	Disclosure Required
CA-1	Level 3: Encouraged	AI is a reasoning assistant (hypotheses, critique, edge cases), never the final authority; unverified AI output earns little credit.	Section J of the response template: tool, purpose, key suggestion, verification, and Correct/Partial/Incorrect/Incomplete reflection.
CA-2	Level 3: Encouraged	AI may suggest hypotheses, parameters, metrics and visualizations; AI-generated numerical results are never experimental evidence.	Report section 17 (AI Disclosure): tool/version, purpose, suggestion, verification, reflection.
CA-3	Level 3: Encouraged	AI acts as a debugging assistant (explain errors, suggest causes, test cases); every diagnosis and fix must be independently verified.	AI record table in the submission (tool, prompt/purpose, important suggestion, verification, Correct/Partial/Incorrect); cases where AI misdiagnosed must be noted.
CA-4	Level 3: Encouraged	AI may assist keyword/search strategy, explanation, critique and starter code; AI-generated references must never be accepted without checking the original publication (Paper → Claim → Evidence → Interpretation).	Mandatory disclosure statement in the report (section 18); if AI was not used, this is stated explicitly.
CA-5	Level 3: Encouraged	AI may suggest commands, code, /proc explanations and debugging hypotheses; AI is not an authority on the state of the student's machine, so each command is understood, executed safely and verified.	Disclosure statement in the submission package (item 13); commands reviewed for safety and observations independently verified.
CA-6	Level 3: Encouraged	AI may help with Rust syntax, compiler errors, test scaffolding, debugging hypotheses and trace analysis; students remain responsible for every line of kernel code, and AI-generated low-level code is not copied blindly.	AI usage disclosure in the final report (section 20) and submission package (item 12); `unsafe` blocks documented independently of AI.
CA-7	Level 1: Prohibited (during the viva)	Authentic individual understanding is being verified in real time; AI assistance cannot be used while answering.	Responsible AI use in CA-1 to CA-6 is itself examined through AI-related viva questions (what AI helped with, what was verified, which claim was rejected).
ESA	Level 1: Prohibited	Closed-book written examination assessing independent understanding.	NA

Course Facilitator